CreatorFlow AI

Comprehensive Testing Plan

Document Type: Comprehensive Testing Plan
Project: CreatorFlow AI — AI-Powered Content Optimization Platform
Version: 1.0
Prepared by: Michael Berry
Date: July 2026
Status: Draft — Submitted for Review
Course: Software Engineering II

© 2026 CreatorFlow AI. All rights reserved.

Table of Contents

1. Introduction 3
- 1.1 Project Overview 3
- 1.2 Purpose and Objectives 3
- 1.3 Importance of Testing 3

2. Software Testing Strategies 4
- 2.1 Attributes of a Good Test 4
- 2.2 Black-Box Testing 4
- 2.3 White-Box Testing 5

3. Test Planning 6
- 3.1 Test Plan Overview 6
- 3.2 Testing Objectives 6
- 3.3 User Understanding 6
- 3.4 Rapid Cycle Testing 7

4. Traceability 8
- 4.1 Requirements Traceability Matrix 8
- 4.2 Regression Testing Plan 9

5. Types of Testing 10
- 5.1 Unit Testing 10
- 5.2 Integration Testing 11
- 5.3 System Testing 12
- 5.4 Validation Testing 13

6. Acceptance Testing 14
- 6.1 Acceptance Testing Process 14
- 6.2 Alpha Testing 14
- 6.3 Beta Testing 15
- 6.4 Acceptance Test Cases 15

7. Release Testing 16
- 7.1 Release Testing Plan 16
- 7.2 Performance and Stress Testing 16
- 7.3 Release Checklist 17

8. Testing Steps and Techniques 18
- 8.1 Test Case Design 18
- 8.2 White-Box Techniques 18
- 8.3 Black-Box Techniques 19

9. Test Recordkeeping 20
- 9.1 Test Case Log 20

10. Design Review and Feedback 22
- 10.1 Review Process 22
- 10.2 Findings and Revisions 22

© 2026 CreatorFlow AI. All rights reserved.

1. Introduction

1.1 Project Overview

CreatorFlow AI is a full-stack, AI-powered content optimization platform designed for independent digital content creators operating across YouTube, TikTok, and Instagram. The platform integrates the OpenAI API (GPT-3.5-turbo) to automate high-value content optimization tasks including title generation, description writing, hashtag research, and SEO keyword recommendations. The application is built on a React 18 frontend, a Node.js/Express backend, Firebase Authentication and Firestore for data persistence, and is deployed via Vercel (frontend) and Render (backend).

Development follows an Agile Scrum methodology structured into six two-week sprints targeting a production-ready MVP. The project is developed by a single developer and covers seven user stories (US-01 through US-07) mapped to functional requirements FR-01 through FR-07 in the Software Requirements Specification (SRS).

1.2 Purpose and Objectives

This Comprehensive Testing Plan documents the complete testing strategy for CreatorFlow AI. Its purpose is to define how the application will be verified against its requirements and validated against real user expectations before deployment.

The specific objectives of this plan are:

- Define the testing strategy, approach, and methodology used throughout the project lifecycle
- Specify unit, integration, system, validation, acceptance, and release test cases aligned to all functional requirements
- Establish traceability from every test case back to its originating requirement or user story
- Document regression testing procedures to prevent regressions when components change
- Define acceptance criteria and the process by which the software will be deemed ready for deployment
- Provide a structured recordkeeping framework for test execution results and defect tracking

1.3 Importance of Testing

Why a Testing Plan Must Precede Development

A testing plan defined before development begins serves as the quality contract for the entire project. It forces precise thinking about what 'correct behavior' means for each requirement, surfaces ambiguities in acceptance criteria before they become coding mistakes, and provides an objective standard against which every sprint's output is evaluated. For CreatorFlow AI specifically, testing is especially critical because the system integrates three external services (OpenAI API, Firebase, and deployment platforms) — each of which can fail in ways that are not under the developer's control. Without a structured testing plan, integration failures between these services would only surface at the final deployment stage, when they are most expensive to fix and least time-efficient to diagnose.

Testing Risk Without a Plan	Mitigation via This Testing Plan
OpenAI API integration errors discovered late in development	Integration test cases for API endpoints defined in Section 5.2 — executed from Sprint 4 onward
Authentication regressions introduced by backend changes	Regression test suite defined in Section 4.2 — re-run after every backend modification

© 2026 CreatorFlow AI. All rights reserved.

Testing Risk Without a Plan	Mitigation via This Testing Plan
UI components not meeting functional acceptance criteria	Validation test cases in Section 5.4 derived directly from user story acceptance criteria
Cross-platform or deployment environment failures	Release testing plan in Section 7 includes production environment smoke tests on Vercel + Render

© 2026 CreatorFlow AI. All rights reserved.

2. Software Testing Strategies

2.1 Attributes of a Good Test

Every test case in this plan is designed against the following four quality attributes for effective software tests:

Attribute	Definition	Application in CreatorFlow AI
High Error Probability	The test is designed to expose a defect, not confirm correct behavior	Test cases target boundary conditions, invalid inputs, unauthorized access, and API error states — all known high-defect areas
Non-Redundant	Each test case covers a distinct scenario not covered by any other test	The traceability matrix in Section 4.1 maps each requirement to exactly the tests that cover it, preventing duplication
Best of Breed	Among similar tests, the most revealing test is chosen	For OpenAI integration, tests exercise actual API calls with controlled inputs rather than simple stub responses
Appropriate Complexity	Tests are not trivially simple nor unnecessarily complex	Test cases cover one specific behavior per case with a clear expected result — no compound assertions that hide failure causes

2.2 Black-Box Testing

Black-box testing treats the system as an opaque unit and validates outputs against inputs without knowledge of the internal implementation. The tester interacts with the application exactly as a user would.

Black-Box Technique	Description	Applied To
Equivalence Partitioning	Divide inputs into valid and invalid classes; test one representative from each	Form validation (email format, password length, topic input character limits)
Boundary Value Analysis	Test at the exact minimum and maximum limits of input ranges	Topic input: 0 chars (empty), 1 char (minimum), 500 chars (maximum), 501 chars (over limit)
Decision Table Testing	Enumerate combinations of input conditions and their expected outcomes	Platform dropdown + tone dropdown + topic field combinations for title generation
Use Case Testing	Derive tests directly from user story scenarios and acceptance criteria	All 7 user stories (US-01 through US-07) each have a corresponding black-box validation test
Error Guessing	Apply experience and intuition to guess likely error conditions	Empty OpenAI API key, expired Firebase token, lost network connection mid-request

2.3 White-Box Testing

White-box testing uses knowledge of the internal implementation to design tests that ensure every code path, branch, and condition is exercised. Applied primarily to the Node.js/Express backend and React component logic.

© 2026 CreatorFlow AI. All rights reserved.

White-Box Technique	Description	Applied To
Statement Coverage	Ensure every executable statement is reached by at least one test	authMiddleware.js, generateController.js, promptBuilder.js
Branch Coverage	Ensure every true/false branch of every conditional is exercised	Token validation (valid / expired / missing), platform conditional prompt construction
Path Coverage	Identify and test all independent execution paths through a module	openaiService.js: success path, OpenAI error path, network timeout path, rate limit path
Condition Coverage	Test every boolean sub-expression independently	Input validation conditions: topic empty, topic too long, platform not in allowed list, tone not in allowed list
Loop Boundary Testing	Verify loop behavior at 0, 1, and maximum iterations	Result parsing loop in generateController.js when OpenAI returns 0, 1, or 5 title suggestions

© 2026 CreatorFlow AI. All rights reserved.

3. Test Planning

3.1 Test Plan Overview

The CreatorFlow AI test plan is structured to align with the 6-sprint Agile development schedule. Testing activities begin in Sprint 1 and scale in scope with each sprint's deliverables. This ensures defects are caught in the sprint they are introduced rather than deferred to a final testing sprint.

Sprint	Development Focus	Testing Activities
Sprint 1	Project scaffold, routing, layout	Manual smoke test: Vercel deployment renders, routing works, no console errors
Sprint 2	Firebase Auth (registration, login, logout)	Unit tests: auth state management. Black-box: login/logout/register flows. Boundary: password < 8 chars
Sprint 3	Node.js + Express backend, auth middleware	White-box: authMiddleware branch coverage. Integration: POST /api/generate/titles returns 401 without token
Sprint 4	OpenAI integration — all 4 AI generation tools	Integration: all 4 endpoints with valid + invalid inputs. Black-box: result card display, copy function
Sprint 5	Firebase Firestore save/retrieve/delete	Unit: Firestore write/read/delete operations. Integration: save flow from UI through to Firestore
Sprint 6	E2E testing, bug fixes, deployment	Full system test, release testing on Vercel + Render production, cross-browser (Edge), acceptance test run

3.2 Testing Objectives

The following explicit, measurable testing objectives govern this plan:

- All 7 user stories (US-01 – US-07) must have at least one passing validation test case before Sprint 6 submission
- All functional requirements FR-01.1 through FR-07.5 must be covered by at least one test case in the traceability matrix
- Zero critical-severity defects may remain open at the time of final deployment
- All API endpoints must return the correct HTTP status codes for both success and error scenarios
- Firebase Firestore security rules must pass validation using the Firebase Rules Emulator before Sprint 6
- The application must render correctly on both desktop (1280px+) and mobile (375px) viewports
- All protected routes must redirect unauthenticated users to the login page without exception

3.3 Understanding the Users

CreatorFlow AI targets independent content creators who are technically comfortable with web applications but are not software engineers. Key user characteristics that influence test case design:

User Characteristic	Testing Implication
Non-technical users who expect intuitive UI	All error messages must be human-readable (not raw error codes). Test that API error states surface as friendly messages.

© 2026 CreatorFlow AI. All rights reserved.

User Characteristic	Testing Implication
Mobile-first content creators	All UI test cases include a mobile viewport (375px) verification step alongside desktop.
Users who may paste long content into input fields	Boundary value tests include pasting 500+ character inputs into topic fields — system must handle gracefully.
Users who expect fast AI responses	Performance test: AI generation response time must be under 10 seconds for 95% of requests under normal load.
Users who save and return to their work	Session persistence tests verify that saved projects survive page refresh and browser close/reopen.

3.4 Rapid Cycle Testing

Rapid cycle testing embeds testing into every sprint rather than deferring it to a dedicated testing phase. This approach is critical for a solo developer project where late-stage defect discovery has no team buffer to absorb the rework cost.

Rapid Cycle Practice	Implementation in CreatorFlow AI
Test cases written before code	For each Sprint 4–5 feature, the test case (input, expected output, pass criteria) is defined before the implementation begins
Daily smoke test on deployed environment	After each commit to the main branch, the Vercel and Render auto-deploy is manually smoke-tested on the production URL
Sprint retrospective includes defect review	End-of-sprint review includes a test results review: open defects are triaged before the next sprint begins
Technical reviews before testing	Code review of each component's implementation against the SDS design before executing its test cases
Regression run after every change	Any modification to authMiddleware.js, api.js, or Firestore rules triggers the regression test suite in Section 4.2

© 2026 CreatorFlow AI. All rights reserved.

4. Traceability

4.1 Requirements Traceability Matrix

The following matrix maps every functional requirement from the SRS to the test cases that verify it. Every requirement must be covered by at least one test case before the project is considered complete.

FR ID	Requirement Summary	Test Case ID(s)	Test Type	User Story
FR-01.1	Allow new user registration	TC-AUTH-01	Validation	US-01 — User Registration
FR-01.2	Validate email format and min password length	TC-AUTH-02	Black-Box	US-01
FR-01.3	Authenticate via Firebase Auth on login	TC-AUTH-03	Integration	US-02 — User Login
FR-01.4	Display descriptive error messages for invalid credentials	TC-AUTH-04	Black-Box	US-02
FR-01.5	Maintain session state across page refreshes	TC-AUTH-05	System	US-02
FR-01.6	Allow logout and redirect to login page	TC-AUTH-06	Validation	US-02
FR-01.7	Redirect unauthenticated users from protected routes	TC-AUTH-07	System	US-02
FR-02.1	Dashboard shows nav links to all four tools	TC-DASH-01	Black-Box	US-01/02
FR-02.2	Dashboard shows saved projects and history	TC-DASH-02	Integration	US-07
FR-02.3	Dashboard accessible only to authenticated users	TC-DASH-03	System	US-01/02
FR-03.1	Title Generator accepts topic, tone, platform inputs	TC-TITLE-01	Black-Box	US-03
FR-03.2	Returns 3–5 optimized title suggestions via OpenAI	TC-TITLE-02	Integration	US-03
FR-03.3	Titles displayed in separated result cards	TC-TITLE-03	Black-Box	US-03
FR-03.4	One-click copy-to-clipboard on each title	TC-TITLE-04	Black-Box	US-03
FR-03.5	Save action persists title to user project	TC-TITLE-05	Integration	US-03/07

© 2026 CreatorFlow AI. All rights reserved.

FR ID	Requirement Summary	Test Case ID(s)	Test Type	User Story
FR-03.6	Loading state displayed during API call	TC-TITLE-06	Black-Box	US-03
FR-04.1	Description Generator accepts topic and platform	TC-DESC-01	Black-Box	US-04
FR-04.2	Returns platform-specific SEO description	TC-DESC-02	Integration	US-04
FR-04.3	Copy-to-clipboard and save actions on description	TC-DESC-03	Black-Box	US-04
FR-04.4	Loading indicator during API processing	TC-DESC-04	Black-Box	US-04
FR-05.1	Hashtag Generator accepts topic or description	TC-HASH-01	Black-Box	US-05
FR-05.2	Returns 10–20 platform-relevant hashtags	TC-HASH-02	Integration	US-05
FR-05.3	Hashtags displayed as copyable formatted list	TC-HASH-03	Black-Box	US-05
FR-05.4	Copy all or save hashtags to project	TC-HASH-04	Black-Box	US-05
FR-06.1	SEO Optimizer accepts video topic input	TC-SEO-01	Black-Box	US-06
FR-06.2	Returns SEO keywords and recommendations	TC-SEO-02	Integration	US-06
FR-06.3	Recommendations in clear structured format	TC-SEO-03	Black-Box	US-06
FR-06.4	Copy or save SEO suggestions to project	TC-SEO-04	Black-Box	US-06
FR-07.1	Save any generated output to named project	TC-SAVE-01	Integration	US-07
FR-07.2	Saved projects persist in Firestore across sessions	TC-SAVE-02	System	US-07
FR-07.3	View all saved projects from dashboard	TC-SAVE-03	Validation	US-07
FR-07.4	Delete individual saved items	TC-SAVE-04	Integration	US-07

© 2026 CreatorFlow AI. All rights reserved.

FR ID	Requirement Summary	Test Case ID(s)	Test Type	User Story
FR-07.5	Saved items display type, content, timestamp	TC-SAVE-05	Black-Box	US-07

4.2 Regression Testing Plan

Regression tests are re-executed whenever a change is made to a component that other components depend on. The following table defines which regression test suites are triggered by each type of change.

Change Event	Regression Suite to Re-Run	Rationale
Any change to authMiddleware.js	TC-AUTH-03, TC-AUTH-07, TC-TITLE-02, TC-DESC-02, TC-HASH-02, TC-SEO-02	All API endpoints depend on auth middleware — a change could break token validation across all routes
Any change to promptBuilder.js	TC-TITLE-02, TC-DESC-02, TC-HASH-02, TC-SEO-02	Prompt construction affects all four AI generation endpoints
Any change to Firestore security rules	TC-SAVE-01 through TC-SAVE-05, TC-AUTH-07	Rule changes can inadvertently grant or deny access to user data
Any change to api.js (Axios client)	TC-TITLE-02, TC-DESC-02, TC-HASH-02, TC-SEO-02, TC-SAVE-01	All frontend API calls route through api.js — a change affects all tool pages
Any change to AuthContext.jsx	TC-AUTH-03 through TC-AUTH-07, TC-DASH-03	Auth context changes affect session state across the entire app
Firebase SDK version upgrade	Full TC-AUTH-* suite	SDK changes may alter token handling, persistence, or sign-in behavior

© 2026 CreatorFlow AI. All rights reserved.

5. Types of Testing

5.1 Unit Testing

Unit tests verify the behavior of individual functions, modules, and components in isolation. External dependencies (Firebase, OpenAI API) are mocked to ensure tests are fast, deterministic, and not dependent on external service availability.

Test ID	Component Under Test	Test Description	Expected Result	Priority
UT-01	promptBuilder.js	buildPrompt('titles', {topic:'coding', platform:'YouTube', tone:'Casual'}) returns string containing all three inputs	Returned string includes 'coding', 'YouTube', 'Casual'	**High**
UT-02	promptBuilder.js	buildPrompt with empty topic throws a validation error	Throws Error('topic is required')	**High**
UT-03	promptBuilder.js	buildPrompt with invalid platform value throws error	Throws Error with platform list	**Medium**
UT-04	authMiddleware.js	Request with valid Firebase token sets req.uid and calls next()	next() called; req.uid === decoded UID	**High**
UT-05	authMiddleware.js	Request with missing Authorization header returns 401	HTTP 401, JSON error body	**High**
UT-06	authMiddleware.js	Request with expired token returns 401	HTTP 401, 'Token expired' message	**High**
UT-07	authMiddleware.js	Request with malformed token returns 401	HTTP 401, 'Invalid token' message	**High**
UT-08	openaiService.js	Successful API call returns parsed array of strings	Array length >= 1, all strings	**High**
UT-09	openaiService.js	OpenAI API error (500) throws mapped application error	Throws OpenAIError with message	**High**
UT-10	AuthContext.jsx	onAuthStateChanged listener updates user state on login	Context user state equals Firebase user object	**High**
UT-11	AuthContext.jsx	logout() clears user state and redirects to /login	User state null after logout	**High**
UT-12	useGenerate.js	Hook sets loading: true during API call, false on response	Loading state transitions correctly	**Medium**
UT-13	useGenerate.js	Hook sets error state when API call fails	Error state contains error message	**Medium**

© 2026 CreatorFlow AI. All rights reserved.

5.2 Integration Testing

Integration tests verify that components work correctly together — specifically the frontend-to-backend REST API calls, the backend-to-OpenAI connection, and the backend-to-Firebase data operations.

Test ID	Integration Boundary	Test Description	Expected Result	Priority
IT-01	Frontend → Backend	POST /api/generate/titles with valid Firebase token and inputs returns 200 with results array	HTTP 200; results[] length 3–5	**High**
IT-02	Frontend → Backend	POST /api/generate/titles without Authorization header returns 401	HTTP 401; error.code === 'UNAUTHORIZED'	**High**
IT-03	Frontend → Backend	POST /api/generate/descriptions with valid inputs returns 200	HTTP 200; results[0] is non-empty string	**High**
IT-04	Frontend → Backend	POST /api/generate/hashtags with valid inputs returns 200 array of 10–20 hashtags	HTTP 200; results[] length 10–20	**High**
IT-05	Frontend → Backend	POST /api/generate/seo with valid inputs returns structured recommendations	HTTP 200; results contains keyword array	**High**
IT-06	Backend → OpenAI API	generateController passes sanitized prompt to openaiService and returns parsed response	No raw prompt in response; results are clean strings	**High**
IT-07	Backend → Firebase Admin	authMiddleware verifies a real Firebase ID token issued by the project	req.uid matches the Firebase user UID	**High**
IT-08	Frontend → Firestore	Save button click writes GeneratedItem document to Firestore under the correct user path	Document visible at /users/{uid}/projects/{pid}/items/{itemId}	**High**
IT-09	Frontend → Firestore	Saved Projects page loads and displays documents from the correct Firestore path	All saved items from Firestore rendered in UI	**High**
IT-10	Frontend → Firestore	Delete button removes the correct Firestore document	Document no longer exists at its path	**Medium**

© 2026 CreatorFlow AI. All rights reserved.

Test ID	Integration Boundary	Test Description	Expected Result	Priority
IT-11	CORS Boundary	POST from Vercel frontend origin returns correct CORS headers	Access-Control-Allow-Origin matches Vercel URL	**Medium**
IT-12	CORS Boundary	POST from disallowed origin is rejected	HTTP 403 or CORS block in browser	**Medium**

5.3 System Testing

System tests verify that the fully assembled application behaves correctly as an integrated whole across both deployment environments (Vercel + Render). These tests simulate real user workflows end-to-end.

Test ID	System Test Scenario	Steps	Expected Result
ST-01	Full Registration → Dashboard Flow	1. Navigate to /register 2. Enter valid email + password 3. Submit	User created in Firebase Auth; redirected to /dashboard; name displayed in header
ST-02	Login with valid credentials	1. Navigate to /login 2. Enter valid credentials 3. Submit	User authenticated; redirected to /dashboard; session persists on refresh
ST-03	Login with invalid credentials	1. Navigate to /login 2. Enter wrong password 3. Submit	Error message displayed; user remains on /login page
ST-04	Unauthenticated access to protected route	1. Open browser without session 2. Navigate to /dashboard	Automatic redirect to /login; no dashboard content visible
ST-05	Full AI Title Generation Flow	1. Login 2. Navigate to Title Generator 3. Fill all fields 4. Click Generate	Loading state shown; 3–5 title cards rendered; copy buttons functional
ST-06	Full AI Description Generation Flow	1. Login 2. Navigate to Description Generator 3. Fill fields 4. Generate	Description card rendered with copy and save buttons
ST-07	Full AI Hashtag Generation Flow	1. Login 2. Navigate to Hashtag Generator 3. Fill fields 4. Generate	10–20 hashtags rendered; copy-all button functional
ST-08	Full AI SEO Optimization Flow	1. Login 2. Navigate to SEO Optimizer 3. Enter topic 4. Generate	Structured SEO recommendations rendered with copy/save actions
ST-09	Save content and retrieve from Saved Projects	1. Generate titles 2. Save one 3. Navigate to Saved Projects	Saved title appears in Saved Projects list with correct type badge and timestamp
ST-10	Delete saved item persists across session	1. Save an item 2. Delete it 3. Reload page	Deleted item does not reappear after page reload

© 2026 CreatorFlow AI. All rights reserved.

Test ID	System Test Scenario	Steps	Expected Result
ST-11	Session persistence across browser refresh	1. Login 2. Refresh browser	User remains authenticated; /dashboard accessible without re-login
ST-12	Mobile viewport rendering	1. Open app on 375px viewport 2. Navigate all screens	All screens render correctly; sidebar collapses; no horizontal scroll

5.4 Validation Testing

Validation tests verify that the software meets the requirements originally specified. Each validation test maps directly to a user story and its acceptance criteria from the SRS, confirming the software does what users actually need.

Test ID	User Story	Acceptance Criterion Tested	Pass Condition
VT-01	**US-01**	User can register with a valid email and password	Firebase user record created; user redirected to dashboard
VT-02	**US-01**	Registration rejected for invalid email or password < 8 chars	Inline error message shown; no Firebase record created
VT-03	**US-02**	User can login and is redirected to dashboard	Firebase session active; /dashboard rendered
VT-04	**US-02**	Incorrect credentials show descriptive error message	'Invalid email or password' message visible; no redirect
VT-05	**US-02**	Logout clears session and returns to login	Firebase session cleared; /login rendered; /dashboard redirects back to /login
VT-06	**US-03**	Title Generator returns 3–5 titles based on topic/platform/tone	API response contains 3–5 non-empty title strings; all rendered as cards
VT-07	**US-04**	Description Generator returns platform-specific description	API response contains description relevant to selected platform; word count reasonable
VT-08	**US-05**	Hashtag Generator returns 10–20 hashtags with # prefix	API response contains 10–20 strings each beginning with #
VT-09	**US-06**	SEO Optimizer returns keywords and actionable recommendations	API response contains keyword list and at least 2 optimization recommendations
VT-10	**US-07**	Generated content can be saved and retrieved	Firestore document created on save; document visible in Saved Projects page
VT-11	**US-07**	Saved items can be deleted	Firestore document deleted; item removed from Saved Projects list

© 2026 CreatorFlow AI. All rights reserved.

Test ID	User Story	Acceptance Criterion Tested	Pass Condition
VT-12	**US-02**	Protected routes inaccessible without authentication	All /dashboard sub-routes redirect to /login when accessed without session

© 2026 CreatorFlow AI. All rights reserved.

6. Acceptance Testing

6.1 Acceptance Testing Process

Acceptance testing is the final stage of verification before deployment. It answers the question: does the software, as deployed, meet the requirements and expectations of its intended users? For CreatorFlow AI, acceptance testing is conducted in Sprint 6 on the production Vercel + Render deployment using real Firebase credentials and a live OpenAI API key.

Acceptance Testing Phase	Timing	Participants	Environment
Alpha Testing	Sprint 6, Week 1	Developer (self-review)	Development environment (localhost + dev Firebase project)
Beta Testing	Sprint 6, Week 1–2	1–2 peer reviewers	Production environment (Vercel + Render + production Firebase)
Formal Acceptance	Sprint 6, Week 2	Course instructor / evaluator	Production environment — final submission state

6.2 Alpha Testing

Alpha testing is conducted by the developer at the development site using the full suite of test cases defined in this plan. The developer acts as a first-line user, following each defined user story flow from start to finish without knowledge of the implementation while executing the tests.

- Alpha testing scope: all 12 system tests (ST-01 through ST-12) and all 12 validation tests (VT-01 through VT-12)
- Alpha testing is conducted on localhost (development environment) before any Sprint 6 changes are deployed to production
- All defects found during alpha testing are logged in the defect register and triaged by severity before beta testing begins
- Alpha testing exit criterion: zero Critical-severity open defects; all High-severity defects have documented workarounds or fix timelines

6.3 Beta Testing

Beta testing releases the application to a small group of real users to identify issues the developer cannot self-detect. For CreatorFlow AI, beta testers are peers from the course or classmates who are given access to the production Vercel deployment.

Beta Testing Activity	Description
Beta tester onboarding	Beta testers receive: the production URL, a test account (email + password), and a short task list matching the user story flows
Task list for beta testers	Register a new account; generate titles for a topic of their choice; save a result; find and delete the saved result; attempt to access /dashboard without logging in
Feedback collection	Beta testers submit feedback via a short structured form: task completed (yes/no), what went wrong, what was confusing, any unexpected behavior

© 2026 CreatorFlow AI. All rights reserved.

Beta Testing Activity	Description
Beta exit criterion	All beta testers complete the core task list without developer assistance. No Critical defects reported.

6.4 Acceptance Test Cases

Test ID	Acceptance Test Description	Pass Criterion	Priority
AT-01	Register a new account using the production URL with a real email address	Account created; redirected to /dashboard; user name displayed	**High**
AT-02	Login to an existing account and verify the dashboard loads with navigation links	Dashboard renders all 4 tool links and Recent Projects section	**High**
AT-03	Generate 3–5 YouTube titles for 'how to start a podcast'	3–5 title cards rendered; each has copy and save buttons	**High**
AT-04	Generate an Instagram description for 'morning workout routine'	Non-empty description rendered; copy and save functional	**High**
AT-05	Generate hashtags for 'travel vlog Tokyo'	10–20 hashtags displayed; copy-all functional	**High**
AT-06	Generate SEO recommendations for 'beginner guitar lessons'	Structured keyword list and recommendations displayed	**High**
AT-07	Save a generated title and navigate to Saved Projects	Saved title appears in list with type badge, content, and timestamp	**High**
AT-08	Delete a saved item and verify it does not reappear after page refresh	Item removed from list; refresh confirms deletion	**Medium**
AT-09	Attempt to access /dashboard in a new browser window without logging in	Redirected to /login; dashboard content not visible	**High**
AT-10	Test on a mobile device or 375px browser viewport	All screens usable; no broken layout; sidebar collapses properly	**Medium**

© 2026 CreatorFlow AI. All rights reserved.

7. Release Testing

7.1 Release Testing Plan

Release testing is the final black-box testing pass conducted on the production deployment immediately before the project is submitted for grading. It verifies that the production environment behaves identically to the tested development environment and that no deployment-specific issues exist.

Release Test Category	Scope	Method
Smoke Testing	Verify the application loads on the production Vercel URL without errors	Navigate to production URL; check no 404 or 500 errors in browser console
Authentication Smoke Test	Login, dashboard render, and logout on production Firebase project	Execute AT-01 and AT-02 on production URL
AI Generation Smoke Test	Execute all four generation tools with a simple test topic	Execute AT-03 through AT-06 with topic: 'test topic for release verification'
Firestore Save/Delete	Save and delete a generated item on production Firestore	Execute AT-07 and AT-08 on production environment
Route Guard Verification	Confirm protected routes redirect unauthenticated users	Execute AT-09 on production URL
Cross-Browser Verification	Verify application renders correctly on Microsoft Edge	Repeat AT-01 through AT-09 on Microsoft Edge (SRS requirement TR-04)
Mobile Viewport	Verify layout on 375px viewport	Execute AT-10 using Chrome DevTools mobile emulation

7.2 Performance and Stress Testing

Performance testing validates that the system meets its non-functional response time requirements under expected usage conditions.

Test ID	Performance Test	Acceptance Threshold	Method
PT-01	AI generation response time under normal load	**< 10 seconds for 95% of requests**	Manual stopwatch timing across 10 consecutive generation requests
PT-02	Page load time for /dashboard on first load	**< 3 seconds on standard broadband**	Browser DevTools Network tab — measure DOMContentLoaded
PT-03	Login response time (Firebase Auth round-trip)	**< 2 seconds from submit to dashboard redirect**	Manual timing across 5 login attempts
PT-04	Firestore save operation response time	**< 2 seconds from button click to success confirmation**	Manual timing across 5 save operations

© 2026 CreatorFlow AI. All rights reserved.

Test ID	Performance Test	Acceptance Threshold	Method
PT-05	Stress test — 10 consecutive rapid generation requests	**No 429 Rate Limited errors; no UI crash**	Submit 10 generation requests in rapid succession; observe behavior
PT-06	OpenAI API budget consumption check	**< $5 per 100 test generation calls**	Check OpenAI dashboard usage after full test run

7.3 Release Checklist

The following checklist must be fully completed and all items checked before the project is submitted as final:

✓	Release Criterion	Verified By
☐	All unit tests (UT-01 through UT-13) passed	Developer self-review
☐	All integration tests (IT-01 through IT-12) passed	Developer self-review
☐	All system tests (ST-01 through ST-12) passed	Developer self-review
☐	All validation tests (VT-01 through VT-12) passed	Developer self-review
☐	All acceptance tests (AT-01 through AT-10) passed on production	Alpha + beta testing
☐	Zero Critical-severity open defects	Defect register review
☐	All High-severity defects resolved or have accepted workarounds	Defect register review
☐	Performance tests (PT-01 through PT-06) pass all thresholds	Manual timing results
☐	Microsoft Edge compatibility verified (SRS TR-04)	Cross-browser test run
☐	Mobile viewport (375px) layout verified	DevTools mobile emulation
☐	Firestore Security Rules validated via Firebase Rules Emulator	Firebase console
☐	OpenAI API key not present in any frontend bundle or repository	DevTools Network + GitHub repo audit
☐	GitHub README includes setup instructions	README review
☐	Environment variables configured correctly on Vercel and Render	Deployment console review

© 2026 CreatorFlow AI. All rights reserved.

8. Testing Steps and Techniques

8.1 Test Case Design

Each test case in this plan was designed following a consistent structure before code development began for the corresponding feature. This design-first approach ensures test cases reflect the intended behavior from the specification rather than the implemented behavior.

Design Step	Description	Application
1. Identify the requirement	Select the specific FR or user story acceptance criterion the test will verify	Every test case references its source requirement in the traceability matrix (Section 4.1)
2. Define the input	Specify the exact input values including valid, invalid, and boundary cases	Each test case specifies exact input values — not vague descriptions like 'some topic'
3. Define the expected output	State the precise expected result before executing the test	Expected results are unambiguous — e.g., 'HTTP 200 with results[] length 3–5', not 'works correctly'
4. Define the pass criterion	State the binary condition that determines test pass or fail	Pass criteria are binary: either the exact expected output matches, or the test fails
5. Identify error conditions	Anticipate what could go wrong and add error path test cases	For every happy-path test, at least one error-path test case exists (e.g., IT-01 has a corresponding IT-02)

8.2 White-Box Testing Techniques

White-box tests for CreatorFlow AI focus on the Node.js/Express backend, where the most complex branching logic resides. The following techniques are applied to the highest-risk modules.

Module	Paths Tested	Coverage Target
authMiddleware.js	Path 1: valid token → next(). Path 2: missing header → 401. Path 3: expired token → 401. Path 4: malformed token → 401	100% branch coverage — all 4 paths exercised by UT-04 through UT-07
promptBuilder.js	Path 1: all valid inputs → prompt string. Path 2: empty topic → throw. Path 3: invalid platform → throw. Path 4: invalid tone → throw	100% branch coverage — all 4 paths exercised by UT-01 through UT-03
openaiService.js	Path 1: success → parsed results. Path 2: OpenAI 500 error → mapped error. Path 3: network timeout → timeout error. Path 4: rate limit 429 → rate limit error	4 paths; UT-08, UT-09 cover paths 1–2; PT-05 covers path 4
generateController.js	Path 1: valid request → 200. Path 2: validation fail → 400. Path 3: auth fail → 401. Path 4: OpenAI error → 500	All paths covered by IT-01 through IT-06

8.3 Black-Box Techniques

© 2026 CreatorFlow AI. All rights reserved.

Black-box techniques are applied to all user-facing features where correctness is judged from the user's perspective, not the implementation. The following techniques are applied throughout the test suite.

Technique	Test Cases Using This Technique	Example
Equivalence Partitioning	TC-AUTH-01, TC-AUTH-02, TC-AUTH-04	Valid email partition: 'user@example.com'. Invalid email partition: 'notanemail'. Each representative tested once.
Boundary Value Analysis	TC-AUTH-02, TC-TITLE-01, TC-DESC-01	Password: 7 chars (fail), 8 chars (pass), 100 chars (pass). Topic: 0 chars (fail), 1 char (pass), 500 chars (pass), 501 chars (fail).
Use Case Testing	VT-01 through VT-12	Each validation test follows the exact step sequence defined in the corresponding user story (US-01 through US-07).
Error Guessing	IT-02, IT-11, IT-12, UT-05–UT-07	Expired token submission, missing Authorization header, request from disallowed CORS origin, empty response from OpenAI.
Decision Table Testing	TC-TITLE-01, TC-DESC-01	Platform (YouTube/TikTok/Instagram) × Tone (Professional/Casual/Hype) × Topic (valid/invalid) combinations tested for title generation.

© 2026 CreatorFlow AI. All rights reserved.

9. Test Recordkeeping

All test executions are recorded in the master test log below. Each entry captures the test ID, description, input values used, expected result, actual result, pass/fail status, and any defect notes. This log is updated after each test execution session.

9.1 Test Case Log

The following log covers a representative subset of executed test cases from the Sprint 1–3 completed features. Sprint 4–6 test results will be appended as testing progresses.

Test ID	Description	Input	Expected	Actual / Status	Result
UT-04	authMiddleware with valid token	Valid Firebase ID token in Authorization header	req.uid set; next() called	Confirmed — uid matches Firebase user	**Pass**
UT-05	authMiddleware missing header	No Authorization header	HTTP 401	HTTP 401 returned	**Pass**
UT-06	authMiddleware expired token	Expired Firebase ID token	HTTP 401	HTTP 401 returned	**Pass**
UT-07	authMiddleware malformed token	'Bearer INVALID_TOKEN'	HTTP 401	HTTP 401 returned	**Pass**
IT-02	POST without auth token	POST /api/generate/titles, no token	HTTP 401	HTTP 401, {error:{code:'UNAUTHORIZED'}}	**Pass**
IT-11	CORS from Vercel origin	POST from Vercel production URL	CORS headers present	Access-Control-Allow-Origin: [vercel-url]	**Pass**
IT-12	CORS from disallowed origin	POST from localhost:3001 (not configured)	CORS block	Request blocked by browser CORS policy	**Pass**
ST-04	Unauthenticated /dashboard access	Navigate to /dashboard without session	Redirect to /login	Redirect fired immediately; dashboard not rendered	**Pass**
ST-11	Session persistence on refresh	Login, then F5 refresh	User still authenticated	Firebase Auth session restored from IndexedDB	**Pass**
VT-02	Invalid email registration	Email: 'notanemail', password: 'test'	Inline validation error	Error shown: 'Please enter a valid email address'	**Pass**
VT-02b	Password too short	Email: valid, password:	Inline error: min 8 chars	Error shown: 'Password	**Pass**

© 2026 CreatorFlow AI. All rights reserved.

Test ID	Description	Input	Expected	Actual / Status	Result
		'1234567' (7 chars)		must be at least 8 characters'	
VT-05	Logout clears session	Click logout button	Redirected to /login; session cleared	Redirect to /login; /dashboard redirects back to /login	**Pass**
TC-AUTH-07	ProtectedRoute guard	Direct URL navigation to /dashboard without auth	Redirect to /login	Redirect fires before any dashboard content renders	**Pass**
PT-01 (partial)	AI title generation time	Topic: 'JavaScript tips for beginners'	< 10 seconds	Avg 4.2 seconds across 5 trials	**Pass**
PT-02	Dashboard page load	Navigate to /dashboard after login	< 3 seconds	DOMContentLoaded: 1.8 seconds	**Pass**

9.2 Defect Register

All defects discovered during testing are logged below. Severity is rated Critical (blocks release), High (significant impact), Medium (notable but workaround exists), or Low (cosmetic / minor).

Defect ID	Severity	Description	Found In	Status	Resolution
DEF-001	**Low**	Description Generator occasionally returns content slightly over platform character limit for TikTok (2200 chars)	IT-03 / Sprint 4	**Open**	Fix: add max_tokens constraint to promptBuilder.js description prompt
(No additional defects at this time)	—	Log will be updated as Sprint 4–6 testing proceeds	—	—	—

© 2026 CreatorFlow AI. All rights reserved.

10. Design Review and Feedback

10.1 Review Process

A structured self-review of this testing plan was conducted following completion of the initial draft. As a solo developer, the review process used a two-pass methodology: a completeness pass (does every required section exist and cover its topic fully?) and a quality pass (are test cases specific enough to be binary pass/fail? Are all requirements covered in the traceability matrix?).

The review evaluated the plan against three standards: the assignment rubric, the functional requirements in the SRS, and real-world testing best practices for web applications integrating third-party APIs.

10.2 Findings and Revisions

Finding	Severity	Revision Applied
Initial traceability matrix only covered FR-01 through FR-03 — FR-04 through FR-07 were missing	High	Traceability matrix in Section 4.1 expanded to cover all 33 sub-requirements across FR-01 through FR-07
Unit tests initially did not include authMiddleware error path coverage (expired and malformed tokens)	High	UT-06 and UT-07 added specifically for expired and malformed token paths in authMiddleware.js
No CORS testing was included in the initial draft	Medium	IT-11 and IT-12 added to integration test suite to verify CORS header behavior from allowed and disallowed origins
Performance tests were not defined — only functional tests	Medium	Section 7.2 added with 6 performance test cases (PT-01 through PT-06) including response time thresholds
Regression testing plan did not specify which change events trigger which test suites	Medium	Section 4.2 revised to include a change-event-to-test-suite mapping table with rationale for each trigger
Beta testing process was described generically without a concrete task list for beta testers	Low	Section 6.3 revised to include a specific 5-task list for beta testers and a structured feedback collection process
Test recordkeeping section had no defect register	Low	Section 9.2 defect register added with severity classification, status, and resolution tracking columns

Post-Review Assessment

Following the review and revision cycle, all 33 functional requirement sub-items from the SRS are traceable to at least one test case in this plan. All six testing types required by the assignment (unit, integration, system, validation, acceptance, release) are represented with specific, binary-pass/fail test cases. The plan is considered complete and ready for submission.

— End of Testing Plan —

© 2026 CreatorFlow AI. All rights reserved.